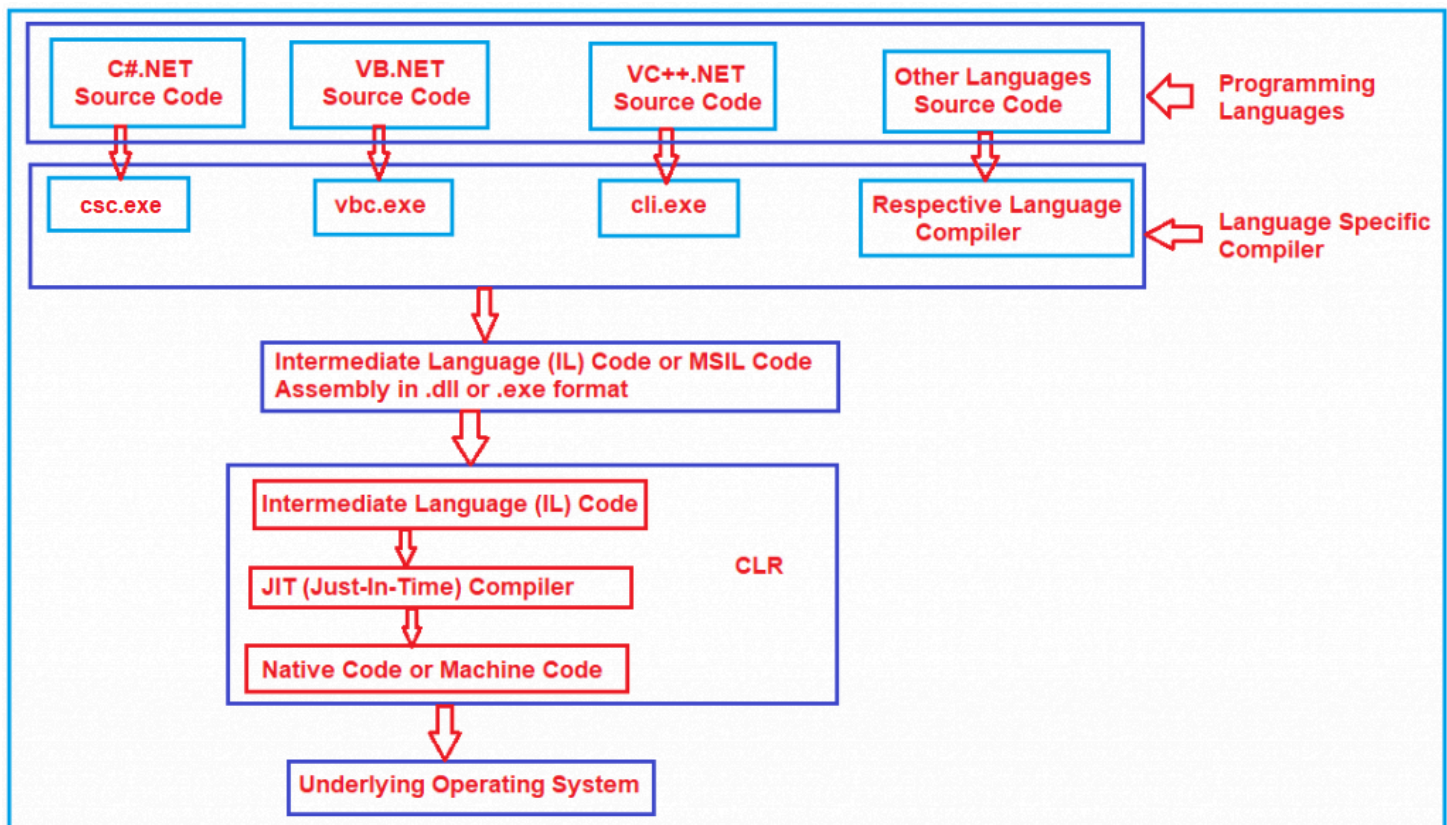


# Common Language Runtime (CLR) in .NET Framework:

In this article, I am going to discuss the **Common Language Runtime (CLR) in .NET Framework**. Please read our previous article before proceeding to this article where we gave a brief introduction to the **DOT NET Framework**. At the end of this article, you will understand all about CLR with Examples. But before understanding Common Language Runtime in .NET Framework, let us first understand how a .NET application is compiled and run.

## How is a .NET Application Compiled and Run?

In order to understand how exactly a .NET Application is compiled and run, please have a look at the following image.



First, the developer has to write the code using any dot net-supported programming languages such as C#, VB, J#, etc. Then the respective language compiler will compile the source code and generate something called Microsoft Intermediate Language (MSIL) or Intermediate Language (IL) code. For example, if the programming language is C#, then the compiler is CSC and if the programming language is VB, then the compiler will be VBC. This Intermediate Language (IL) code is half-compiled code i.e. partially compiled code and cannot be executed directly by the Operating System. To execute this Microsoft Intermediate Language (MSIL) or Intermediate Language (IL) code on our machine, the .NET Framework provides something called Common Language Runtime (CLR) which takes the responsibility to execute our Microsoft Intermediate Language (MSIL) or Intermediate Language (IL) Code.

The CLR takes the IL (Intermediate Language) code and gives it to something called the JIT (Just-in-Time) Compiler which is part of the CLR. The JIT compiler reads each and every line of the IL code and converts it to machine-specific instructions (i.e. into binary format) which will be executed by the underlying Operating System. The CLR in .NET Framework provides the runtime environment to execute the .NET Applications.

## What is Intermediate Language (IL) Code in .NET Framework?

The Intermediate Language or **IL code** in .NET Framework is a half-compiled or partially-compiled or CPU-independent partially compiled code and this code can not be executed by Operating System.

## Why Partial Compiled Code or Why Not Fully Compiled Code?

As a developer, you might be thinking about why the respective language compiler generates partially compiled code or why not fully compiled code i.e. machine code or binary code in .NET Framework when we compile the source code. The reason is very simple. We don't know in what kind of environment .NET Code is going to be run (for example, Windows XP, Windows 7, Windows 10, Windows 11, Windows Server, etc.).

In other words, we don't know what operating system is going to run our application. We also don't know the CPU configuration, Machine Configuration, Security Configuration, etc. of the corresponding operating system where we want to run our .NET Applications. So, the Microsoft Intermediate language (MSIL) or Intermediate language (IL) code is partially compiled, and at runtime, this Microsoft Intermediate language (MSIL) or Intermediate language (IL) code is compiled to Machine-Specific instructions or you can say binary code based on underlying Operating System, CPU, Machine Configuration, etc. by the CLR in .NET Framework.

## Common Language Runtime (CLR) in .NET Framework:

CLR is the heart of the .NET Framework and it contains the following components.

1. Security Manager
2. JIT Compiler
3. Memory Manager
4. **Garbage Collector**
5. **Exception Manager**
6. **Common Language Specification (CLS)**
7. **Common Type System (CTS)**

Let us discuss what each of these components does in detail in .NET Framework.

### Security Manager:

There are basically two components to managing security. They are as follows:

1. **CAS (Code Access Security)**
2. **CV (Code Verification)**

These two components are basically used to check the privileges of the current user and then check whether the user is allowed to access the assembly or not. The Security Manager also checks what kind of rights or authorities this code has and whether it is safe to be executed by the Operating System or not. So, basically, these types of security checks are managed by the Security Manager Component of CLR in .NET Application.

### JIT Compiler:

The JIT (Just-In-Time) Compiler is responsible for Converting the MSIL code into Native Code (Machine Code or Binary code) that is going to be executed by the Operating System. The Native Code (Machine Code or Binary code) is directly understandable by the system hardware. JIT compiles the code just before the execution and then saves this translation in memory.

## Memory Manager:

The Memory Manager component of CLR in the .NET Framework allocates the necessary memory for the variables and objects that are to be used by the application.

## Garbage Collector:

When a dot net application runs, lots of objects are created. At a given point in time, it is possible that some of those objects are not used by the application. So, **Garbage Collector in .NET Framework** is nothing but a **Small Routine** or you can say it's a **Background Process Thread** that runs periodically and try to identify what objects are not being used currently by the application and de-allocates the memory of those objects.

## Exception Manager:

The **Exception Manager** component of CLR in the .NET Framework redirects the control to execute the catch or finally blocks whenever an exception has occurred at runtime. If we have not handled the Runtime Exception, then the Exception Manager will throw the exception and abnormally terminate the program execution at the line where the exception has occurred.

## Common Type System (CTS) in .NET Framework:

The .NET Framework supports many programming languages such as C#, VB.NET, J#, etc. Every programming language has its own data type. One programming language data type cannot be understood by other programming languages. But, there can be situations where we need to communicate between two different programming languages. For example, we need to write code in the VB.NET language and that code may be called from C# language. In order to ensure smooth communication between these languages, the most important thing is that they should have a **Common Type System (CTS)** which ensures that data types defined in two different languages get compiled to a common data type.

CLR in .NET Framework will execute all programming language's data types. This is possible because CLR has its own data types which are common to all programming languages. At the time of compilation, all language-specific data types are converted into CLR's data type. This data type system of CLR is common to all .NET Supported Programming languages and this is known as the **Common Type System (CTS)**.

## CLS (Common Language Specification) in .NET Framework:

**CLS (Common Language Specification)** is a part of CLR in the .NET Framework. The .NET Framework supports many programming languages such as C#, VB.NET, J#, etc. Every programming language has its own syntactical rules for writing the code which is known as language specification. One programming language's syntactical rules (language specification) cannot be understood by other programming languages. But, there can be situations where we need to communicate between two different programming languages. In order to ensure smooth communication between different .NET Supported Programming Languages, the most important thing is that they should have **Common Language Specifications** which ensures that language specifications defined in two different languages get compiled into a Common Language Specification.

CLR in .NET Framework will execute all programming language's code. This is possible because CLR has its own language specification (syntactical rules) which are common to all .NET Supported Programming Languages. At the time of compilation, every language compiler should follow this language specification of CLR and generate the MSIL code. This language specification of CLR is common for all programming languages and this is known as **Common Language Specifications (CLS)**.

In the next article, I am going to discuss the **.NET Program Execution** process. Here, in this article, I try to explain the Common Language Runtime (CLR) in C#.NET Framework. I hope you enjoy this CLR in C# article.